

- PROFILE

Frontend Engineer

더존비즈온(코스피 상장 B2B SaaS)에서 **3년 3개월간** AI 기능 프론트엔드와 React Native 앱을 담당. 권한 설계 · AI 상태 흐름 · WebView Bridge를 실서비스에서 직접 다뤘습니다.

- 일하는 방식

Problem first. Screen next. Technology last.

DOMAINS

AI Productivity · React Native Mobile · Enterprise Collaboration

SPECIALTIES

AI 응답 상태 흐름 · WebView ↔ Native Bridge · 권한 설계

CORE STACK

React · React Native · TypeScript · Redux Toolkit · Context API

PERSONAL PRODUCTS

WhatFlix (Movie Community) · Pixel Garden Portfolio

— SUMMARY

경력 요약 · 한눈에 보는 업무 범위

더존비즈온(코스피 상장 B2B SaaS)에서 3년 3개월간 — AI 기능 UI 통합 · React Native 모바일 앱 · B2B 협업 SaaS의 Frontend 영역을 담당. 기술을 먼저 고르지 않고, 사용자 문제와 운영 비용을 함께 고려한 구조 설계를 우선합니다.

— PROJECT HIGHLIGHTS

NO.	PROJECT	담당 업무 요약	STACK
01	Enterprise AI Productivity 기업 AI 생산성 SaaS · Web + Mobile	AI Chat 스트리밍 UI · OCR / STT 화면 흐름 · Prompt Library · AI 회의록 뷰어 · AI 기능 공통 진행/실패 상태 이름 정리.	React · React Native · Context API · Native Module · AI API
02	Enterprise Mobile Platform React Native 기반 기업 모바일 앱	WebView ↔ Native Bridge 메시지 프로토콜 · Camera · Gallery · 파일 접근 · Deep Link · 권한 요청 흐름 · 플랫폼 분기 공용 유틸.	React Native · TypeScript · WebView Bridge · Native Module
03	Enterprise Collaboration Platform B2B 협업 SaaS · 대기업/중견기업 임직원	문서 공유 · 조직도 트리 · 역할별 권한 관리 UI · 댓글 · 공통 선택 컴포넌트 추출 · 권한 판단 Custom Hook 진입점 설계.	React · TypeScript · Redux Toolkit · REST · WebSocket
04	Personal Products WhatFlix · Pixel Garden Portfolio	기획 · 디자인 · 개발 · 운영을 직접 소유. 외부 API · AI 추천 · 커뮤니티 · 인터랙티브 UI 설계까지 직접 다뤘습니다.	React · TypeScript · Spring Boot · Pixel UI

Enterprise AI Productivity Platform

반복 업무를 AI 기능으로 대체해 임직원 생산성을 높이는 통합 AI SaaS — **AI API를 안정적인 사용자 경험으로 연결**하는 Frontend 영역 담당.

서비스 대상	담당 역할	팀 구성	사용 기술
기업 임직원	FE · React Native Engineer	기획 · AI · FE · Native · BE	React · React Native
Web · Mobile 양쪽	AI 기능 UI 통합 / Native 연동	운영 단계 서비스	Context API · AI API · Native Module

— 담당 화면 화면 6개 · AI 기능 UI 통합

AI Chat · 스트리밍 질문 ... AI 응답 F-01 · AI 채팅	OCR · Camera F-02 · OCR	STT · Voice F-03 · STT	Prompt 즐겨찾기 F-04 · 프롬프트	AI 회의록 ▶ 요약 ▶ 액션 F-05 · 회의록	중요문서함 F-06 · 중요문서함
---	---	--	---	--	--

— 운영 중 마주한 문제 · 결정 · 결과

문제 AI 응답이 길어질수록 화면이 느려졌다 AI Chat의 긴 응답 도중 다른 UI 입력이 늦게 반영되고, 화면을 떠났다 돌아오면 진행 상태가 모호하다는 피드백이 반복. OCR · STT 실패 화면은 "다시 시도" 버튼 하나뿐이어서 사용자가 다음 행동을 알기 어려웠습니다.	결정 받는 단위와 보여주는 단위 분리 응답을 받는 단위와 화면을 갱신하는 단위를 분리해 흐르는 체감은 유지하되 다른 UI에 영향을 덜 주도록 했습니다. AI Chat · OCR · STT 전체가 같은 진행/실패 상태 이름을 공유하고, 화면 이탈 시 응답 수신을 한 곳에서 정리합니다. flush 주기는 디자인 · 기획과 체감 테스트 후 합의.	결과 긴 응답 중 다른 UI 반응성이 흔들리는 현상 해소 — 디자인 · 기획 체감 테스트 통과. 같은 진행/실패 이름 공유로 신규 AI 기능을 붙일 때 화면별로 처음부터 다시 짤 일 감소. QA 회귀 시나리오가 "상태 이름 단위"로 수렴해 검증 면적이 줄었습니다.
--	---	---

가장 어려웠던 의사결정

throttle만 적용하면 "AI가 멈춘 것 같다"는 피드백이 나왔고, 전부 모아서 한 번에 보여주면 흐름감이 사라졌습니다. 결국 **"흐름감"과 "안정성"의 절충값**을 찾는 일이 가장 오래 걸렸습니다.

배운 점

AI 기능에서는 응답을 보여주는 일보다 **기다림과 실패를 다루는 일**이 더 큰 비중을 차지했습니다. "성공 화면"보다 먼저 "기다림 · 실패 화면"을 설계하는 습관이 생겼습니다.

Enterprise Mobile Platform

PC에서만 쓰던 협업 · AI 서비스를 모바일에서도 같은 품질로 사용할 수 있도록, **WebView와 Native 기능을 연결하는 Bridge 레이어**를 다룬 React Native 기반 기업 모바일 앱.

서비스 대상	담당 역할	팀 구성	사용 기술
기업 임직원 iOS · Android	React Native Engineer Bridge · Native 연동 담당	기획 · FE · Native · BE 운영 단계 서비스	React Native · TypeScript WebView Bridge · Native Module

— 담당 화면 화면 5개

카메라

촬영 · 저장

M-01 · 카메라

갤러리

M-02 · 갤러리

웹뷰

{ web }

Web 화면 임베드

M-03 · 웹뷰

권한 요청

승인

거부

설정 이동

M-04 · 권한

Native Bridge

W ↔ N

M-05 · 브리지

— 운영 중 마주한 문제 · 결정 · 결과

문제 런타임이 다른 두 세계 WebView 안의 Web 코드는 Native 기능에 직접 닿을 수 없었고, Android와 iOS의 파일 접근·권한 처리 방식도 달랐습니다. OS 분기 코드가 화면 로직에 섞이면서 같은 종류의 버그가 화면마다 다시 나타났습니다.	결정 경계 레이어 위치를 먼저 정한다 메시지 프로토콜은 기능 단위로 타입을 나눠 수신 측이 명시적으로 검증. 플랫폼 차이는 화면 외부의 공용 유틸 레이어에서 처리하고, 권한 요청은 일관된 상태 흐름으로 정리했습니다.	결과 - 새 Native 기능 추가 시 구현 범위가 명확해지고, 화면 코드에서 OS 분기가 사라졌습니다. - 디버깅 위치가 한 곳으로 모여면서 같은 종류의 버그 재발이 줄었습니다. - iOS·Android에서 같은 기능 흐름을 같은 코드로 다룰 수 있게 됐습니다.
---	--	---

가장 어려웠던 의사결정

메시지 프로토콜을 너무 세분화하면 규약이 자주 바뀌었고, 너무 추상화하면 디버깅이 어려웠습니다. **"확장 가능하면서 명시적인"** 균형점을 찾는 일이 가장 오래 걸렸습니다.

배운 점

크로스플랫폼 작업에서는 OS 차이를 화면 코드가 직접 다루지 않도록 **경계 레이어의 위치를 먼저 정하는 것**이 이후 기능 추가 비용을 절감한 다는 점을 배웠습니다.

Enterprise Collaboration Platform

문서 공유 · 권한 관리 · 조직도 · 댓글 · 메신저 · 화상회의를 단일 플랫폼으로 통합한 B2B 협업 SaaS — 대기업 · 중견기업 임직원이 매일 사용하는 운영 단계 서비스.

서비스 대상	담당 역할	팀 구성	사용 기술
대기업·중견기업 임직원 B2B Enterprise SaaS	Frontend Engineer 협업 기능 영역 단독 개발	기획 · 디자인 · FE · BE 전사 운영 단계 서비스	React · TypeScript Redux Toolkit · WebSocket

— 담당 화면

화면 6개

문서 공유 · 링크

권한

만료

F-01 · 링크 발급

조직도 트리

본부

팀 A

· 인원 ①

F-02 · 조직도

역할별 권한

관리자

편집자

뷰어

F-03 · 권한 관리

댓글 · 다중 선택

@멘션

F-04 · 댓글

메신저

...

실시간

F-05 · 메신저

화상회의

F-06 · 화상회의

— 운영 중 마주한 문제 · 결정 · 결과

문제 권한 정책이 자주 바뀌었다 운영 단계에서 새 기능보다 권한 정책 변경이 더 자주 발생했고, 같은 정책이 여러 화면에 흩어져 있어 같은 수정을 반복해야 했습니다. 화면마다 별도로 권한 상태를 들고 있어 화면 간 결과가 어긋나는 일도 있었습니다.	결정 권한 판단을 화면 바깥으로 Context · HOC · 서버 only를 검토한 뒤, 화면이 사용자/역할 비교를 직접 하지 않고 "이 기능을 쓸 수 있는지"만 Custom Hook에 묻는 구조로 정리. 권한 변경 이벤트도 한 곳에서 받아 처리합니다. 모든 화면을 한 번에 옮기지 못해 새 기능부터 적용하는 방식으로 점진 이동.	결과 권한 정책 변경 시 수정 범위를 Hook 중심으로 줄였습니다 — 팀 내부 체감 기준 대응 시간 단축. 새 합류 동료가 "권한이 어디서 정해지나"를 한 곳에서 확인 가능. 같은 종류의 회귀 이슈가 재발하지 않게 됐습니다.
---	---	--

가장 어려웠던 의사결정

공통의 범위를 어디까지 정의할 것인가 — 너무 일찍 추상화하면 변형이 어렵고, 너무 늦으면 부채가 굳습니다. 같은 패턴이 다른 맥락에서 반복되는 것을 먼저 확인한 뒤 Hook으로 추출하는 기준을 팀과 합의했습니다.

배운 점

운영에서는 새 기능보다 **정책 변경**이 더 자주 발생합니다. 이후에는 화면을 그리기 전에 권한 기준이 어디에 있어야 하는지를 먼저 정리하고 개발하게 됐습니다.

개인 제품 · 기획부터 운영까지 직접 소유

WhatFlix — Movie Community

"오늘 뭐 볼지 모르겠다"는 일상의 불편에서 출발한 개인 제품. **외부 영화 API · AI 추천 · 커뮤니티 기능**을 직접 기획하고 결합한 풀스택 사이드 프로젝트.

담당 범위	프론트엔드	백엔드	외부 연동
기획 · 디자인 · 개발 · 운영	React · TypeScript	Spring Boot	영화 데이터 API
전 영역 1인 소유	상태 관리 · 라우팅 · UI	REST API · DB 설계 · 인증	+ AI 추천 모델 활용

— 기술 결정 배경 · 왜 이 선택을 했는가

01 FRONTEND React + TypeScript 본업에서 다루던 스택 그대로 — 개인 프로젝트의 학습 비용을 줄이고 "운영 가능한 코드"를 빠르게 만드는 데 집중. 라이브러리 선택보다 폴더 구조와 컴포넌트 경계 정리에 시간을 더 썼습니다.	02 BACKEND Spring Boot 직접 구축 BaaS(Firebase 등)도 검토했지만, REST 설계·인증·DB 스키마를 처음부터 직접 다뤄보는 경험이 더 필요하다고 판단해 Spring Boot로 결정. 프론트 입장에서만 보던 백엔드의 사결정의 무게를 직접 체감했습니다.	03 AI 추천 "사람의 언어가 섞인 추천" 기존 OTT 추천이 시청 기록 위주라면, 여기에 사용자 코멘트와 평점 텍스트를 추천 입력으로 함께 사용. 본업에서 "AI API를 화면에 연결"하던 경험을 반대 방향(직접 호출 · 결과 가공)에서도 다뤄보고 싶었습니다.
---	--	---

— 소유자 입장에서 배운 점

기능을 늘리는 것보다 줄이는 결정이 더 어렵다.

혼자 모든 결정을 내리다 보니 "있으면 좋은" 기능이 핵심 경험을 가리는 경우를 자주 마주쳤고, 그때마다 무엇을 빼야 하는지가 가장 어려웠습니다.

프론트 · 백 · DB 의사결정이 서로 영향을 준다.

본업에서는 API 스펙을 받아 쓰는 입장이었지만, 직접 만들어보며 "이 응답 구조가 화면을 얼마나 단순하게 만들 수 있는지"를 양쪽에서 동시에 생각하게 됐습니다.

— 한계 / 남은 일

AI 추천 결과의 품질 검증을 충분히 하지 못했고, 사용자 규모가 작아 추천이 잘 작동하는지를 정량적으로 평가하기 어려웠습니다. 다음에 같은 시도를 한다면 **평가 지표 설계를 먼저** 만들어두겠습니다.

— STACK

React	TypeScript	Spring Boot	REST API	영화 API 연동	AI 추천
-------	------------	-------------	----------	-----------	-------

— PERSONAL PRODUCT · 작은 카드

Pixel Garden — Interactive Portfolio

단순 이력 페이지가 아니라, **게임형 UI와 인터랙션**을 활용해 개발 경험을 하나의 제품처럼 전달하는 인터랙티브 포트폴리오.

— 만든 이유

포트폴리오 자체가 곧 자기소개 형식이 될 수 있다.

일반적인 포트폴리오는 텍스트와 스크린샷의 나열에 그쳐 개발자의 성향과 인터랙션 감각이 잘 전달되지 않습니다. 픽셀 그래픽과 게임형 인터랙션을 차용해 **"읽는" 포트폴리오가 아니라 "움직여 보는" 포트폴리오**를 만들어 보고 싶었습니다.

— 배운 점

전달 매체의 형식이 전달되는 내용을 바꿀 수 있다.

방문자가 페이지를 단순 스크롤하는 것이 아니라 직접 조작하며 콘텐츠를 발견하는 흐름의 사용자 경험을 만들었고, 인터랙션 자체가 자기소개 일부가 될 수 있다는 점을 배웠습니다.

— STACK

React

TypeScript

Pixel UI

Game-style Interaction

— EXPERIENCE SUMMARY

이런 경험을 가지고 있습니다.

01 AI 기능(AI Chat · OCR · STT) UI 통합 경험

AI API를 화면에 연결·통합하면서, 응답 지연·실패·취소 상황에서도 사용자가 현재 진행 상태를 이해할 수 있도록 UI를 개선했습니다.

02 React Native 기반 모바일 서비스 개발 경험

WebView Bridge · Native Module · Camera · 권한 요청 흐름 · Deep Link · 플랫폼 분기를 직접 구현했고, OS 차이를 화면 외부의 공용 유틸 레이어에서 다루도록 정리했습니다.

03 Enterprise SaaS 협업 플랫폼 개발 경험

문서 공유 · 조직도 · 역할별 권한 · 댓글 · 메신저 · 화상회의 영역을 단독 개발했고, 반복되는 권한 정책 변경 속에서 유지보수성을 높이기 위한 구조 조정을 진행했습니다.

04 사용자 경험과 유지보수성을 고려한 구조 설계 경험

Custom Hook · 공통 컴포넌트 · 메시지 프로토콜처럼, 같은 패턴이 다른 맥락에서 반복되는 것을 확인한 뒤 추출하는 식으로 조기 추상화의 부채를 피해 왔습니다.

05 다양한 직군과 협업하며 제품을 개선한 경험

기획 · 디자인 · AI · Native · Backend와 메시지 타입, 권한 모델 같은 경계 규약을 먼저 합의하고, 그 위에서 운영 단계의 제품을 함께 개선했습니다.

— IN SUMMARY

기술이 아니라 문제와 화면으로 일하는 개발자.

주요 도메인

AI Productivity (Web · Mobile)

React Native Mobile

Enterprise Collaboration

개인 제품

WhatFlix · Pixel Garden Portfolio

읽어 주셔서 감사합니다. 더 자세한 화면 설계 과정과 의사결정 트레이드 오프는 인터뷰에서 이어서 나누고 싶습니다.